

DOCUMENTO DE PLANEACIÓN Y ESPECIFICACIÓN TÉCNICA

**Proyecto: FitTrack CLI - Sistema de Gestión Integral para
Gimnasios**

Fecha de elaboración: 29 de septiembre de 2026

Versión del documento: 1.0

Tecnologías: Node.js, MySQL2, Programación Orientada a Objetos (POO), Patrones de Diseño.

1. DOCUMENTO DE PLANEACIÓN SCRUM

1.1. Roles Asignados

Rol	Responsabilidades Principales
Product Owner (PO)	Define los requisitos del negocio, prioriza el Product Backlog, acepta los entregables de cada Sprint y asegura que el producto aporte valor.
Scrum Master (SM)	Facilita las ceremonias Scrum, elimina impedimentos del equipo, asegura el cumplimiento de la metodología y la calidad del proceso.
Developer(s)	Responsable(s) de la implementación del código, pruebas unitarias/integración, aplicación de patrones de diseño y documentación técnica.

1.2. Sprints Definidos

Sprint	Duración	Objetivo Principal
Sprint 1	Día 1	Fundación: Análisis, arquitectura, configuración de BD y estructura del proyecto.
Sprint 2	Día 2	Core Business: Gestión de clientes, planes de entrenamiento y contratos automáticos.
Sprint 3	Día 3	Funcionalidades Avanzadas: Seguimiento físico con rollback, nutrición y finanzas con transacciones.
Sprint 4	Día 4	Entrega Final: Pruebas, correcciones, documentación final y video demo.

1.3. Épicas del Proyecto

- **Épica 1: Fundación (Día 1):** Análisis del problema, definición de arquitectura y patrones, diseño de base de datos MySQL y estructura de carpetas del proyecto.
 - **Épica 2: Core Business (Día 2):** Desarrollo de los módulos de gestión de clientes, gestión de planes de entrenamiento y generación automática de contratos.
 - **Épica 3: Funcionalidades Avanzadas (Día 3):** Implementación de seguimiento físico con rollback, módulo de nutrición y gestión financiera con transacciones SQL reales.
 - **Épica 4: Entrega Final (Día 4):** Ejecución de pruebas, corrección de bugs, documentación completa y grabación del video demo.
-

2. TAREAS DETALLADAS CON SUBTAREAS

TASK 01 — Análisis del Problema (Día 1 - 1 hora)

- **Identificar problema principal:** Definir "Gestión integral de clientes de gimnasio/entrenador personal". Documentar en README.
- **Identificar usuarios:** Primario (Entrenador/Dueño), Secundario (Administrador).
- **Identificar necesidades:** CRUD de clientes, planes, seguimiento físico, nutrición, control financiero y contratos automáticos.
- **Identificar restricciones:** Node.js + MySQL + mysql2, POO + SOLID + 2 patrones de diseño, 4 días de desarrollo.
- **Identificar operaciones críticas:** Transacciones financieras, cancelación con rollback, generación automática de contratos.

TASK 02 — Definir Alcance (Día 1 - 30 min)

- **Funcionalidades obligatorias (MVP):** CRUD clientes/planes, contratos automáticos, seguimiento físico, planes nutricionales básicos, registro de ingresos/egresos, transacciones en pagos, rollback en cancelación.
- **Funcionalidades secundarias:** Reportes financieros avanzados, gráficos de progreso, exportación de datos.
- **Funcionalidades fuera del MVP:** App móvil, integración con pasarelas de pago reales, multi-tenant, IA para recomendaciones.
- **Entregable final:** Código en GitHub, README completo, PDF de Scrum, video demo (< 7 min), BD funcional.

TASK 03 — Definir Módulos (Día 1 - 30 min)

- **Cientes:** Modelo, Repository, Service, Command (validaciones: edad 14-99, email).
- **Planes:** Modelo, Repository, Service, Command (nombre, duración, metas, nivel).
- **Contratos:** Modelo, Repository, Service (generación automática), integración con PlanService.
- **Seguimiento:** Modelo, Repository, Service (rollback crítico), Command.
- **Nutrición:** Modelo (NutritionPlan, FoodLog), Repository, Service, Command.
- **Finanzas:** Modelo, Repository (transacciones SQL), Service, Command.

TASK 04 — Diseñar Arquitectura (Día 1 - 1 hora)

- **Estructura de carpetas:** `src/` (config, models, repositories, services, commands, factories, utils), `database/`, `docs/`, `tests/`.
- **Responsabilidades:** Models (validaciones), Repositories (acceso a datos), Services (lógica de negocio), Commands (interfaz CLI), Factories (creación de objetos).
- **Flujo CLI:** Usuario → `app.js` → Menú → CommandFactory → Command → Service → Repository → MySQL.

TASK 05 — Definir Patrones de Diseño (Día 1 - 30 min)

- **Repository Pattern:** Separar lógica de acceso a datos de la lógica de negocio (`src/repositories/`).
- **Factory Pattern:** Crear objetos complejos (contratos automáticos, comandos CLI) (`src/factories/`).
- **Command Pattern:** Encapsular acciones CLI como objetos con método `execute()` (`src/commands/`).
- **Documentación:** Diagrama de arquitectura y comentarios `// PATRÓN: [Nombre]` en el código.

TASK 06 — Configurar MySQL y Conexión (Día 1 - 1 hora)

- Instalar dependencias (`mysql2`, `dotenv`, `chalk`, `readline-sync`).
- Crear `.env` y `src/config/db.config.js` (pool de conexiones).
- Crear scripts `database/init.sql` (tablas) y `database/seed.sql` (datos de prueba).
- Probar conexión con script `test-db.js`.

TASK 07 — Implementar Modelos con Validaciones (Día 1 - 1 hora)

- Crear `BaseModel.js` y `validator.js` (`isRequired`, `isNumber`, `isEmail`, etc.).
- Implementar modelos específicos: `Client.js`, `Plan.js`, `Contract.js`, `FitnessRecord.js`, `NutritionPlan.js`, `FoodLog.js`, `FinancialTransaction.js` con sus respectivas validaciones de dominio.

TASK 08 — Implementar Repositories (Día 1 - 1 hora)

- Crear `BaseRepository.js` (`findAll`, `findById`, `create`, `update`, `delete`).
- Implementar repositorios específicos heredando de `BaseRepository`.
- `FinancialRepository`: Incluir método `createWithTransaction` para operaciones críticas.

TASK 09 — Implementar Services (Día 2 - 2 horas)

- `ClientService` y `PlanService`: CRUD y asociación.
- `ContractService`: `generateAutoContract()` y `cancelContract()`.
- `FitnessService`: `registerProgress()` y `rollbackProgress()`.
- `NutritionService`: Planes, registro de alimentos y reporte semanal.
- `FinancialService`: `registerIncome()`, `registerExpense()` con transacciones reales.

TASK 10 — Implementar Commands (CLI) (Día 2 - 1.5 horas)

- Crear `BaseCommand.js` y comandos específicos para cada módulo.
- Implementar `CommandFactory.js` para instanciación dinámica.
- Configurar `src/app.js` como punto de entrada del menú principal.

TASK 11 — Implementar Transacciones Críticas (Día 3 - 2 horas)

- Implementar `beginTransaction()`, `commit()` y `rollback()` en:
 1. `FinancialService.registerIncome/Expense`
 2. `FitnessService.rollbackProgress`
 3. `ContractService.cancelContract` (actualiza contrato + elimina seguimiento).
- Probar casos de éxito y fallo para verificar el rollback.

TASK 12 — Pruebas y Correcciones (Día 3 - 1.5 horas)

- Probar CRUD de clientes, asignación de planes, seguimiento físico, nutrición y finanzas.
- Validar casos límite (edad < 14, error de DB simulado).
- Documentar "Known Issues" si aplica.

TASK 13 — Documentación (Día 4 - 2 horas)

- Completar `README.md` con toda la información técnica, arquitectura y patrones.
- Generar PDF de Scrum (este documento).
- Crear diagrama de arquitectura en `docs/`.

- Agregar comentarios JSDoc en el código.

TASK 14 — Video Demo (Día 4 - 2 horas)

- Grabar video de máximo 7 minutos mostrando: Tablero ClickUp, arquitectura, modelos, demo de contrato automático, demo de cancelación con rollback y demo de finanzas.
- Subir a YouTube/Drive y agregar el enlace al README.

TASK 15 — Entrega Final (Día 4 - 30 min)

- Verificar repositorio GitHub (código, README, PDF, `.gitignore`, `.env.example`).
- Probar ejecución completa desde cero (`init.sql` + `seed.sql` + `node src/app.js`).
- Entregar enlaces al profesor/cliente.

3. REQUERIMIENTOS Y HISTORIAS DE USUARIO

ID	Historia de Usuario	Criterios de Aceptación
HU-01	Como entrenador personal, quiero crear, listar, actualizar y eliminar clientes, para mantener una base de datos organizada de los usuarios del gimnasio.	El sistema valida que la edad esté entre 14 y 99 años y que el correo tenga un formato válido.
HU-02	Como administrador del gimnasio, quiero que al asociar un cliente a un plan se genere automáticamente un contrato, para formalizar la relación comercial sin pasos manuales.	El contrato incluye condiciones, precio, fechas y se vincula al <code>plan_cliente_id</code> en la misma transacción.
HU-03	Como entrenador, quiero registrar el progreso físico de mis clientes, y que si el plan se cancela, estos registros se eliminen, para mantener la consistencia e integridad de la base de datos.	La cancelación del plan ejecuta una transacción que actualiza el estado del contrato a 'cancelado' y elimina los registros de <code>seguimiento_fisico</code> .

HU-04	Como nutricionista, quiero crear planes de alimentación y registrar alimentos con sus calorías, para generar un reporte nutricional semanal del cliente.	El reporte agrupa las calorías por tipo de comida (desayuno, almuerzo, cena, merienda) en los últimos 7 días.
HU-05	Como gerente financiero, quiero registrar ingresos y egresos utilizando transacciones de base de datos reales, para garantizar que no existan inconsistencias en los pagos.	Cualquier fallo durante el registro provoca un ROLLBACK automático. El balance se calcula como (Ingresos - Gastos).

4. HERRAMIENTA DE SEGUIMIENTO Y EVIDENCIAS

- **Herramienta de Gestión de Proyectos:** ClickUp
- **Integración:** Cada commit incluye el ID de tarea de ClickUp (ej. `feat(FT-11): implement financial transactions`). Automatizaciones configuradas: `push` → "In Progress", `merge` → "Complete".
- **Evidencia en PDF entregada en el repositorio:** [Enlace al repositorio GitHub / Carpeta docs/scrum-plan.pdf] (*Reemplazar con el enlace real*)
- **Video Demo:** [Enlace al video de YouTube / Google Drive] (*Reemplazar con el enlace real*)

5. ANEXO A: DOCUMENTACIÓN TÉCNICA DEL PROYECTO (README)

FitTrack CLI - Sistema de Gestión Integral

Proyecto en **Node.js + MySQL2** que implementa un sistema de gestión integral para entrenadores personales y gimnasios. Desarrollado aplicando **Programación Orientada a**

Objetos (POO), principios **SOLID**, patrones de diseño (**Repository**, **Factory**, **Command**) y transacciones **SQL** reales.

Estructura del Proyecto

```
fittrack/
├── database/
│   └── database.sql
├── docs/
│   └── requerimientos.md
├── src/
│   ├── funciones.js
│   ├── main.js
│   ├── menu.js
│   └── transacciones.js
├── .git/
├── .gitignore
├── node_modules/
├── package.json
├── package-lock.json
├── README.md

```

Acciones Críticas y Consistencia de Datos

Las operaciones críticas utilizan `beginTransaction()`, `commit()` y `rollback()` para garantizar la integridad:

Acción Crítica	Descripción	Ubicación en Código
Registrar pago/gasto	Inserta transacción financiera. Si falla, rollback automático.	<code>FinancialService.registerIncome/Expense()</code>
Asignar plan con contrato	Crea el registro <code>plan_cliente</code> y el <code>contrato</code> en una sola transacción.	<code>PlanService.assignPlanToClient() + ContractService</code>
Cancelar plan con rollback	Cancela el contrato y elimina los registros de seguimiento físico del cliente.	<code>ContractService.cancelContract() + FitnessService</code>

Patrones de Diseño y SOLID Aplicados

1. **Repository Pattern:** Separa el acceso a datos de la lógica de negocio. Facilita el testing y el cambio de motor de BD.
2. **Factory Pattern:** Encapsula la creación de objetos complejos (ej. `ModelFactory.createContract()` calcula fechas y condiciones automáticamente).
3. **Command Pattern:** Encapsula las acciones de la CLI como objetos independientes con un método `execute()`, facilitando la escalabilidad del menú.
4. **SOLID:**
 - **S:** Cada clase tiene una única responsabilidad (Model, Repository, Service, Command).
 - **O:** `BaseRepository` permite extender funcionalidades sin modificar su código base.
 - **L:** Todos los repositorios son intercambiables gracias a la herencia de `BaseRepository`.
 - **I:** Interfaces segregadas para lectura y escritura en repositorios.
 - **D:** Inyección de dependencias en los Services (ej. `ClientService` recibe `ClientRepository`).